rqbit HTTP API 라우팅 및 핸들러 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 HTTP API 구현을 분석합니다. rqbit은 Axum 웹 프레임워크를 사용하여 RESTful API를 제공하며, 토렌트 관리, 파일 스트리밍, DHT 통계, 웹 UI 등의 기능을 HTTP 엔드포인트로 노출합니다.

Axum 기반 라우팅 구조

1. 메인 HTTP API 구조체

```
pub struct HttpApi {
api: Api,
opts: HttpApiOptions,
}
```

```
#[derive(Debug, Default)]
pub struct HttpApiOptions {
pub read_only: bool,
pub basic_auth: Option<(String, String)>,
pub allow_create: bool,
#[cfg(feature = "prometheus")]
pub prometheus_handle: Option<metrics_exporter_prometheus::PrometheusHandle>,
```

```
}
```

`HttpApi` 는 rqbit의 HTTP 서버를 관리하는 메인 구조체입니다:

- `api` : 핵심 비즈니스 로직을 담당하는 `Api` 인스턴스
- `opts` : HTTP API 설정 옵션들 (읽기 전용 모드, 인증, 토렌트 생성 허용 등)

2. 라우터 구성

메인 라우터 생성 (`make_api_router`)

```
pub fn make_api_router(state: ApiState) -> Router {
let mut api_router = Router::new()
.route("/", get(h_api_root))
.route("/stream_logs", get(logging::h_stream_logs))
.route("/rust_log", post(logging::h_set_rust_log))
.route("/dht/stats", get(dht::h_dht_stats))
.route("/dht/table", get(dht::h_dht_table))
.route("/stats", get(torrents::h_session_stats))
.route("/torrents", get(torrents::h_torrents_list))
// ... 더 많은 라우트들
}
```

라우터는 다음과 같은 구조로 구성됩니다:

읽기 전용 엔드포인트들:

- `GET /` : API 목록 및 서버 정보

- `GET /dht/stats` : DHT 통계 정보
- `GET /dht/table` : DHT 라우팅 테이블
- `GET /stats` : 전역 세션 통계
- `GET /torrents` : 토렌트 목록
- `GET /torrents/{id}` : 특정 토렌트 상세 정보
- `GET /torrents/{id}/stats/v1` : 토렌트 통계
- `GET /torrents/{id}/stream/{file_id}` : 파일 스트리밍

쓰기 가능 엔드포인트들 (read_only가 false일 때만):

```
if !state.opts.read_only {
api_router = api_router
.route("/torrents", post(torrents::h_torrents_post))
.route("/torrents/{id}/pause", post(torrents::h_torrent_action_pause))
.route("/torrents/{id}/start", post(torrents::h_torrent_action_start))
.route("/torrents/{id}/forget", post(torrents::h_torrent_action_forget))
.route("/torrents/{id}/delete", post(torrents::h_torrent_action_delete))
// ... 기타 POST 엔드포인트들
}
```

3. 미들웨어 스택

CORS 설정

```
let cors_layer = {
```

```
use tower_http::cors::{AllowHeaders, AllowOrigin};


const ALLOWED_ORIGINS: [&[u8]; 4] = [
b"http://localhost:3031", // Webui-dev
b"http://127.0.0.1:3031",
b"http://localhost:1420", // Tauri dev
b"tauri://localhost", // Tauri prod
];


tower_http::cors::CorsLayer::default()
.allow_origin(AllowOrigin::predicate(move |v, _| {
ALLOWED_ORIGINS.contains(&v.as_bytes())
|| allow_regex
.as_ref()
.map(move |r| r.is_match(v.as_bytes()))
.unwrap_or(false)
}))
.allow_headers(AllowHeaders::any())
};
```

기본 인증 미들웨어

```
async fn simple_basic_auth(
expected_username: Option<&str>,
```

```
expected_password: Option<&str>,

headers: HeaderMap,

request: axum::extract::Request,

next: Next,

) -> Result<axum::response::Response> {

// Base64 디코딩을 통한 기본 인증 처리

let user_pass = headers

.get("Authorization")

.and_then(|h| h.to_str().ok())

.and_then(|h| h.strip_prefix("Basic "))

.and_then(|v| base64::engine::general_purpose::STANDARD.decode(v).ok())

.and_then(|v| String::from_utf8(v).ok());

// ... 인증 로직

}
```

트레이싱 미들웨어

```
.layer(

tower_http::trace::TraceLayer::new_for_http()

.make_span_with(|req: &Request| {

let method = req.method();

let uri = req.uri();

if let Some(ConnectInfo(addr)) = req.extensions().get::<ConnectInfo<_>>() {
```

```
debug_span!("request", %method, %uri, addr=%addr.0)
} else {
debug_span!("request", %method, %uri)
}
})
.on_response(DefaultOnResponse::new().include_headers(true))
.on_failure(/* ... */)
)
```

핸들러 구현 분석

1. 토렌트 관련 핸들러들

토렌트 목록 조회 (`h_torrents_list`)

```
pub async fn h_torrents_list(
State(state): State<ApiState>,
Query(opts): Query<ApiTorrentListOpts>,
) -> impl IntoResponse {
axum::Json(state.api.api_torrent_list_ext(opts))
}
```

- Axum의 `State` 추출기를 사용하여 애플리케이션 상태에 접근
- `Query` 추출기로 쿼리 파라미터를 파싱
- JSON 응답으로 토렌트 목록 반환

토렌트 추가 (h_torrents_post)

```
pub async fn h_torrents_post(
State(state): State<ApiState>,
Query(params): Query<TorrentAddQueryParams>,
Timeout(timeout): Timeout<600_000, 3_600_000>,
data: Bytes,
) -> Result<impl IntoResponse> {
// 데이터 형식 추론 (URL vs 토렌트 파일)
let maybe_magnet = |data: &[u8]| -> bool {
std::str::from_utf8(data)
.ok()
.and_then(|s| Magnet::parse(s).ok())
.is_some()
};
let add = match is_url {
Some(true) => AddTorrent::Url(/* ... */),
Some(false) => AddTorrent::TorrentFileBytes(data.into()),
None if SUPPORTED_SCHEMES.iter().any(|s| data.starts_with(s.as_bytes()))
|| maybe_magnet(&data) => AddTorrent::Url(/* ... */),
_ => AddTorrent::TorrentFileBytes(data.into()),
};
tokio::time::timeout(timeout, state.api.api_add_torrent(add, Some(opts)))
```

```
.await
.context("timeout")?
.map(axum::Json)
}
```

특징:

- 커스텀 `Timeout` 추출기를 사용한 타임아웃 처리
- 자동 데이터 형식 감지 (마그넷 링크, URL, 토렌트 파일)
- 비동기 처리와 타임아웃 적용

2. 스트리밍 핸들러

파일 스트리밍 (`h_torrent_stream_file`)

```
pub async fn h_torrent_stream_file(
State(state): State<ApiState>,
Path(StreamPathParams { id, file_id, .. }): Path<StreamPathParams>,
headers: http::HeaderMap,
) -> Result<impl IntoResponse> {
let mut stream = state.api.api_stream(id, file_id)?;
let mut status = StatusCode::OK;
let mut output_headers = HeaderMap::new();
// Range 헤더 처리
if let Some(range) = headers.get(http::header::RANGE) {
let offset: Option<u64> = range
```

```
.to_str().ok()
.and_then(|s| s.strip_prefix("bytes="))
.and_then(|s| s.strip_suffix('-'))
.and_then(|s| s.parse().ok());
if let Some(offset) = offset {
status = StatusCode::PARTIAL_CONTENT;
stream.seek(SeekFrom::Start(offset)).await.context("error seeking")?;
// Content-Range 헤더 설정
}
}
// DLNA 지원
const DLNA_TRANSFER_MODE: &str = "transferMode.dlna.org";
if headers.get(DLNA_TRANSFER_MODE)
.map(|v| matches!(v.as_bytes(), b"Streaming" | b"streaming"))
.unwrap_or(false) {
output_headers.insert(DLNA_TRANSFER_MODE, HeaderValue::from_static("Streaming"))
}
}
```

특징:

- HTTP Range 요청 지원 (부분 콘텐츠 다운로드)
- DLNA 프로토콜 지원
- MIME 타입 자동 감지

- 비동기 파일 시킹

3. DHT 관련 핸들러들

DHT 통계와 라우팅 테이블 정보를 제공하는 엔드포인트들이 구현되어 있습니다.

오류 처리 및 상태 코드 관리

1. ApiError 구조체

```
#[derive(Debug)]

pub struct ApiError {

status: Option<StatusCode>,

kind: ApiErrorKind,

}


#[derive(thiserror::Error, Debug)]

pub enum ApiErrorKind {

#[error("torrent not found {0}")]

TorrentNotFound(TorrentIdOrHash),

#[error("DHT is disabled")]

DhtDisabled,

#[error("unauthorized")]

Unauthorized,

#[error("{0}")]
```

```
Text(&'static str),

#[error(transparent)]

OtherAnyhow(#[from] anyhow::Error),

// ... 기타 오류 타입들

}
```

2. HTTP 응답 변환

```
impl Serialize for ApiError {

fn serialize<S>(&self, serializer: S) -> Result<S::Ok, S::Error> {

#[derive(Serialize, Default)]

struct SerializedError<'a> {

error_kind: &'a str,

human_readable: String,

status: u16,

status_text: String,

#[serde(skip_serializing_if = "Option::is_none")]

id: Option<TorrentIdOrHash>,

}

// ... 직렬화 로직

}

}
```

3. 상태 코드 매핑

- `404 NOT_FOUND` : 토렌트를 찾을 수 없음, DHT 비활성화
- `401 UNAUTHORIZED` : 인증 실패
- `206 PARTIAL_CONTENT` : Range 요청 성공
- `500 INTERNAL_SERVER_ERROR` : 기타 내부 오류

요청/응답 타입 시스템

1. 쿼리 파라미터 처리

```rust
#[derive(Serialize, Deserialize, Default)]
pub struct TorrentAddQueryParams {
pub overwrite: Option<bool>,
pub output_folder: Option<String>,
pub sub_folder: Option<String>,
pub only_files_regex: Option<String>,
pub only_files: Option<OnlyFiles>,
pub peer_connect_timeout: Option<u64>,
pub peer_read_write_timeout: Option<u64>,
pub initial_peers: Option<InitialPeers>,
pub is_url: Option<bool>,
pub list_only: Option<bool>,
}
```

2. 커스텀 직렬화/역직렬화

```
impl<'de> Deserialize<'de> for OnlyFiles {
fn deserialize<D>(deserializer: D) -> core::result::Result<Self, D::Error> {
let s = String::deserialize(deserializer)?;
let list = s.split(',')
.try_fold(Vec::<usize>::new(), |mut acc, c| match c.parse() {
Ok(i) => { acc.push(i); Ok(acc) }
Err(_) => Err(D::Error::custom(format!(
"only_files: failed to parse {:?} as integer", c
))),
})?;
Ok(OnlyFiles(list))
}
}
```

타임아웃 처리

커스텀 Timeout 추출기

```
pub struct Timeout<const DEFAULT_MS: usize, const MAX_MS: usize>(pub Duration);
```

```
impl<S, const DEFAULT_MS: usize, const MAX_MS: usize> axum::extract::FromRequest
```

```
for Timeout<DEFAULT_MS, MAX_MS> {

async fn from_request_parts(parts: &mut Parts, _state: &S) -> Result<Self, Self:

// 쿼리 파라미터 또는 헤더에서 타임아웃 값 추출

let timeout_ms = q.timeout_ms

.map(Ok)

.or_else(|| {

parts.headers.get("x-req-timeout-ms")

.map(|v| std::str::from_utf8(v.as_bytes())

.context("invalid utf-8 in timeout value"))

.map(|v| v.and_then(|v| v.parse::<usize>()

.context("invalid timeout integer")))

})

.transpose()?

.unwrap_or(DEFAULT_MS);

let timeout_ms = timeout_ms.min(MAX_MS);

Ok(Timeout(Duration::from_millis(timeout_ms as u64)))

}

}
```

특징:

- 컴파일 타임 기본값과 최대값 설정
- 쿼리 파라미터와 HTTP 헤더 모두 지원
- 자동 범위 제한

웹 UI 통합

정적 파일 서빙

```
pub fn make_webui_router() -> Router {
Router::new()
.route("/", get(|| async {
([("Content-Type", "text/html")],
include_str!("../../webui/dist/index.html"))
}))
.route("/assets/index.js", get(|| async {
([("Content-Type", "application/javascript")],
include_str!("../../webui/dist/assets/index.js"))
}))
// ... 기타 정적 파일들
}
```

특징:

- 컴파일 타임에 정적 파일들을 바이너리에 임베딩
- 적절한 MIME 타입 설정
- 개발/프로덕션 환경 모두 지원

보안 고려사항

1. 기본 인증

- Base64 인코딩된 사용자명/비밀번호 검증
- WWW-Authenticate 헤더를 통한 브라우저 인증 프롬프트

2. CORS 정책

- 허용된 오리진 목록 관리
- 정규식을 통한 동적 오리진 허용
- 개발 환경과 프로덕션 환경 구분

3. 읽기 전용 모드

- 상태 변경 엔드포인트들을 조건부로 비활성화
- 안전한 읽기 전용 API 제공

성능 최적화

1. 비동기 처리

- 모든 핸들러가 async/await 패턴 사용
- Tokio 런타임을 통한 효율적인 I/O 처리

2. 스트리밍 최적화

- HTTP Range 요청을 통한 부분 다운로드
- 비동기 파일 시킹
- 메모리 효율적인 스트림 처리

3. 타임아웃 관리

- 요청별 커스터마이징 가능한 타임아웃

- 최대 타임아웃 제한을 통한 리소스 보호

결론

rqbit의 HTTP API는 Axum 프레임워크를 기반으로 한 현대적이고 효율적인 구현을 보여줍니다. 주요 특징은:

1. **모듈화된 핸들러 구조**: 기능별로 분리된 핸들러 모듈들
2. **강력한 타입 시스템**: Rust의 타입 시스템을 활용한 안전한 API
3. **유연한 미들웨어**: 인증, CORS, 로깅 등의 횡단 관심사 처리
4. **스트리밍 지원**: HTTP Range 요청과 DLNA 프로토콜 지원
5. **포괄적인 오류 처리**: 구조화된 오류 타입과 적절한 HTTP 상태 코드
6. **성능 최적화**: 비동기 처리와 효율적인 리소스 관리

이러한 설계는 BitTorrent 클라이언트의 복잡한 요구사항을 만족시키면서도 확장 가능하고 유지보수하기 쉬운 API를 제공합니다.